

Command Structure

20190701/20200310/202007/202111 pd

November 2021 Update for discussion of impedance measurement

e-chip2 is programmed through a sequence of serially-transmitted words, which have the structure shown below. There is an 8-bit data field, a 4-bit command field and a 1-bit token field. Although there are 4 command bits, only 2 are used. Those 2 bits steer the 8 data bits into one of four registers:

- The data register
- The row and column registers
- The command register

Each pixel has an 8-bit register that sets the gain, the mode (CLAMP or not) and whether the plating current is on sinking or sourcing. The RESET command sets all pixels to Gain 100, CLAMP mode and no plating current.

Under normal circumstances, programming the chip(s) means

1. Executing a RESET
2. Passing the TOKEN
3. Programming all of the global registers (see annotated programming example at end)
4. Programming all of the (512) pixel registers
5. (repeat steps 2-4 for all other chips)
6. Pass NOT TOKEN
7. Execute a SYNC command
8. Execute the GO command (and start recording data)

For impedance measurement of a single channel (I think):

1. Execute a RESET
2. Pass the TOKEN
3. Program all of the global registers (see annotated programming example at end)
4. Execute a SYNC command
5. Execute the GO command (and start recording data)
6. Program the desired pixel to plate UP
7. Send N DAC values that cover the positive half of a sine wave
8. Program the desired pixel to plate DOWN
9. Send N DAC values that cover the negative half of a sine wave
10. Loop for desired number of cycles

For the serial register, the assumption is a serial data clock of X MHz. 17 clocks are needed to program one serial word, and there should be a few clock gap between sending serial commands.

Original User Guide from March 2020:

Serial data to e-chip are 16-bit words:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	T	C3	C2	C1	C0	D7	D6	D5	D4	D3	D2	D1	D0
				Command				Data Field							

Send 17 serial clocks as the 16 bits are latched on the 17th clock.

101 is the start bit(s). Data are serialized to the chip from B15 to B14 to ... B0

The Command bits are:

T	C3	C2	C1	C0	
1	x	x	x	x	Token required
0	x	x	x	x	Token not required
	0	0	0	0	Write Data register
	0	0	0	1	Write Row register
	0	0	1	0	Write Column register
	0	0	1	1	Execute command (in D field)
	0	1	x	x	(Reserved)
	1	0	x	x	(Reserved)
	1	1	x	x	(Reserved)

Commands C=0,1,2 transfer the contents of the Data Field to the Data, Row, Column registers

Command C=3 executes the command in the Data Field:

Bit	Execute Command		
0	PassTok		
1		GO	Start
2		SYNC	Sets phase=0 on next 4 ns clock
3	Write		Write Reg addressed by Row & Col
4			
5			
6		DigRST	Digital Reset of full chip
7			
	These commands require the token to be present (TokIn=1, TokOut=0)		
	These are broadcast commands (token not required)		

Digital Reset

T	C	D
0	3	0x40

i.e. serialize out the following bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	1	1	0	1	0	0	0	0	0	0

Digital Reset sets all registers to their default value. Execute once at startup.

Sync

T	C	D
0	3	0x04

i.e. serialize out the following bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	1	1	0	0	0	0	0	1	0	0

Sync lines up the internal clock divider phase with the high-speed serializer. Execute once at startup.

GO

T	C	D
0	3	0x02

i.e. serialize out the following bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	1	1	0	0	0	0	0	0	1	0

GO initiates readout. After Digital Reset, the high-speed serial outputs will send idle patterns. Once GO is issued, pixel ADC data are sent. See below for serialization details.

Registers

e-chip 19 has 16 Rows x 32 Columns. When ROW = 0x00 – 0x0F and COL = 0x00 – 0x1F data are written to the individual pixel registers. Otherwise, when COL is greater than 0x3F (ROW = don't care) data are written to other registers:

Column (Binary)	Registers
0b000x xxxx	Pixel
0b001x xxxx	Reserved
0b010x xxxx	DACs
0b011x xxxx	Clock Patterns, Control Reg
0b1xxx xxxx	Reserved

Control Register

T	C	D	
1	0	--	Write data
1	2	0x78	Write "column" (address)
1	3	0x08	Write to register

i.e. serialize out the following bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	0	0	D	D	D	D	D	D	D	D
1	0	1	1	0	0	1	0	0	1	1	1	1	0	0	0
1	0	1	1	0	0	1	1	0	0	0	0	1	0	0	0

Control Register bits

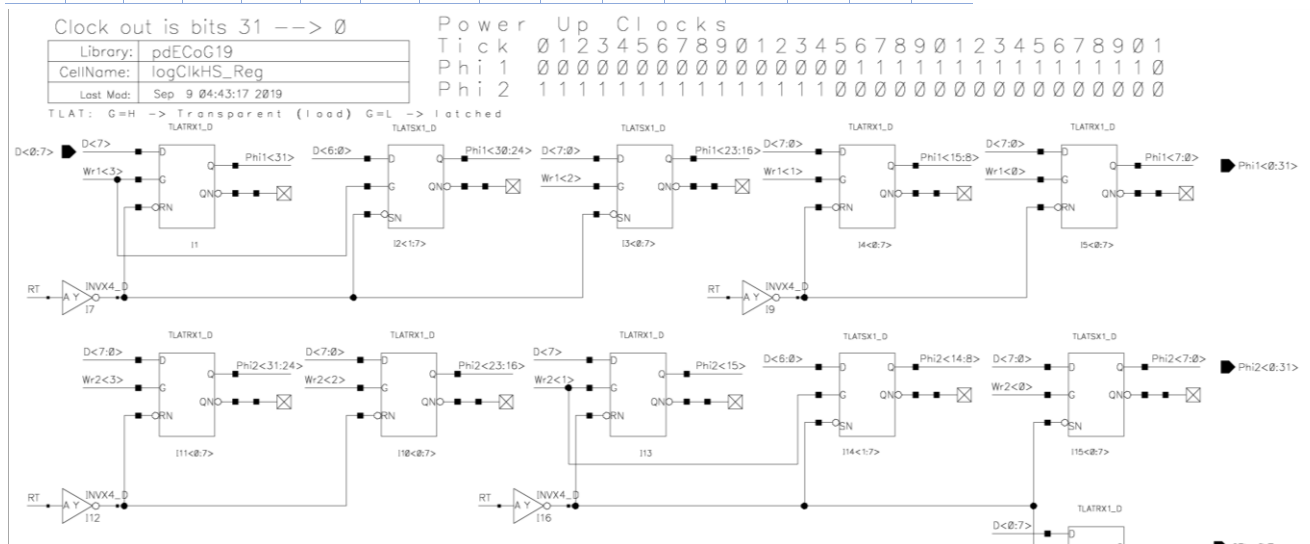
b	Function	RST
7	Reserved	0
6	Reserved	0
5	Reserved	0
4	Reserved	0
3	Reserved	0
2	LVDS Current Boost	0
1	Invert PhiNF w.r.t. PhiN	0
0	Swap Phi1F <-> Phi2F	0

Phi Registers

T	C	D	
1	0	--	Write data
1	2	0x7n	Write "column" (address n=0..7)
1	3	0x08	Write to register

i.e. serialize out the following bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	0	0	D	D	D	D	D	D	D	D
1	0	1	1	0	0	1	0	0	1	1	1	0	n	n	n
1	0	1	1	0	0	1	1	0	0	0	0	1	0	0	0



The non-overlapping Phi clocks default, on reset as above. Write the 8 registers below, if desired, to modify the patterns:

Phi Registers

Write Pattern Bits					Reset (Default)			
Phi1	31..24	23..16	15..8	7..0	01111111	11111111	00000000	00000000
n	3	2	1	0	3	2	1	0
Phi2	31..24	23..16	15..8	7..0	00000000	00000000	01111111	11111111
n	7	6	5	4	7	6	5	4

DACs

DACs

T	C	D	
1	0	--	Write data
1	2	0x4n	Write "column" (address n=0..F)
1	3	0x08	Write to register

n	DAC	n	DAC
0	BiasK1	8	BiasBuf
1	BiasK2	9	Spare (to pad)
2	BiasLVM	A	BiasColDrive
3	LVDSRx	B	LV DSTx
4	BiasPlate	C	BiasLVP
5	BiasK3	D	BiasVC
6	BiasAmp	E	BiasAAF
7	BiasCMP	F	BiasOff

DACs are 5 bits.

Reset value for all DACs = 0x08

Main DAC

T	C	D	
1	0	--	Write data
1	2	0x50	Write "column"
1	3	0x08	Write to register

Bits	Function	RST
0..4	Main DAC Value	0x08
5	Bias Kickstart	1
6	Bias Over-ride (to external pad)	0

Main DAC sets the primary current. The other DACs are referenced to the main current

DAC Options

T	C	D	
1	0	--	Write data
1	2	0x51	Write "column"
1	3	0x08	Write to register

